

Manual de Usuario para Sistema No Relacional

Biblioteca Juvenil

Sistema Web para Biblioteca Escolar

Alumno: Nombre del alumno

Modulo: Desarrollo de aplicaciones web

Semestre: Sexto semestre

Materia: Base de datos no relacionales

Fecha: 10 de junio de 2026

Indice

1. Proposito del sistema
2. Tecnologias utilizadas
3. Como ejecutar el sistema
4. Configuracion necesaria
5. Diagrama de colecciones
6. Base de datos y colecciones
7. Relaciones NoSQL
8. Restricciones de integridad
9. Funcionamiento de los estilos

- 10. Uso del sistema paso a paso
- 11. CRUD paso a paso
- 12. Capturas de pantalla simuladas
- 13. Validaciones y manejo de errores
- 14. Lista de verificación

Manual de Usuario

1. Propósito del Sistema

Biblioteca Juvenil es un sistema web básico para administrar una biblioteca escolar pequeña. El sistema permite registrar autores, libros, usuarios y préstamos. También permite consultar información en tablas, buscar registros, editar datos, actualizar documentos, eliminar registros e imprimir listas.

El propósito principal es facilitar el control de los libros disponibles, los autores registrados, los alumnos que usan la biblioteca y los préstamos activos o entregados.



2. Tecnologías Utilizadas

El proyecto utiliza tecnologías sencillas y adecuadas para una aplicación escolar de base de datos no relacional.

Tecnología		Uso dentro del sistema
HTML		Estructura de las paginas del sistema.
CSS		Estilos visuales personalizados.
Bootstrap local		Diseno de botones, tablas, tarjetas y formularios.
Bootstrap	Icons	Iconos en la barra de navegacion y botones.

local

JavaScript basico	Peticiones al servidor, busquedas, impresion y mensajes.
SweetAlert2 local	Alertas de exito, error y confirmacion.
Node.js	Servidor principal del sistema.
Express	Rutas con app.get y app.post.
MongoDB	Base de datos no relacional.

3. Como Ejecutar el Sistema

Para ejecutar el sistema se debe tener MongoDB activo. Despues se abre una terminal en la carpeta del proyecto y se ejecuta el servidor Node.

```
cd /home/leonardo/biblioteca_juvenil node server.js
```

Cuando el servidor este encendido, se abre el navegador en la direccion:

```
http://localhost:3000
```

El archivo principal es **server.js**. En ese archivo estan las rutas **app.get** para mostrar paginas y consultar datos, y las rutas **app.post** para guardar, actualizar y eliminar informacion.

4. Configuracion Necesaria

Elemento	Configuracion
Servidor	Node.js ejecutando server.js en el puerto 3000.

Base de datos	MongoDB en mongodb://127.0.0.1:27017.
Nombre de la base	biblioteca_juvenil.
Paginas	Carpeta pages.
Archivos publicos	Carpeta public.
Estilos	public/css/estilos.css.
JavaScript	public/js/script.js.
Imagenes	public/img.

5. Diagrama de Colecciones

Como el proyecto usa MongoDB, el diagrama representa colecciones y referencias por ObjectId. No se usan llaves foraneas SQL, sino referencias guardadas en campos.

autores	libros	prestamos	usuarios
_id nombre nacionalidad correo	_id titulo genero anio autorId	_id libroId usuarioId fechaPrestamo fechaEntrega estado	_id nombre telefono correo grupo
Un autor puede estar relacionado con muchos libros.	Un libro pertenece a un autor y puede aparecer en prestamos.	Un prestamo relaciona un libro con un usuario.	Un usuario puede tener varios prestamos.

6. Base de Datos y Colecciones

La base de datos se llama **biblioteca_juvenil**. Contiene cuatro colecciones principales.

Colección	Campos	Descripción
autores	_id, nombre, nacionalidad, correo	Guarda la información básica de los autores.
libros	_id, título, género, año, autorId	Guarda los libros y la referencia del autor.
usuarios	_id, nombre, teléfono, correo, grupo	Guarda los alumnos o usuarios de biblioteca.
prestamos	_id, libroId, usuarioId, fechaPrestamo, fechaEntrega, estado	Guarda los préstamos de libros.



7. Relaciones NoSQL: ObjectId, \$lookup y aggregate

MongoDB crea un campo **_id** para cada documento. El sistema usa **ObjectId** para relacionar documentos entre colecciones.

- **libros.autorId:** guarda el **_id** de autores.
- **prestamos.libroId:** guarda el **_id** de libros.
- **prestamos.usuarioId:** guarda el **_id** de usuarios.

En la lista de prestamos se usa una consulta con **aggregate()**. Esta consulta incluye **\$lookup** para unir prestamos con libros y usuarios. Despues usa **\$unwind** para convertir los arreglos relacionados en objetos sencillos. Finalmente usa **\$project** para mostrar solo campos importantes.

```
prestamos.aggregate([ $lookup con libros, $lookup con  
usuarios, $unwind libro, $unwind usuario, $project  
nombreUsuario, tituloLibro, fechas y estado ])
```

8. Restricciones de Integridad

El sistema usa restricciones sencillas porque es un proyecto escolar con MongoDB. Para registrar un libro se debe seleccionar un autor existente. Para registrar un prestamo se debe seleccionar un libro existente y un usuario existente.

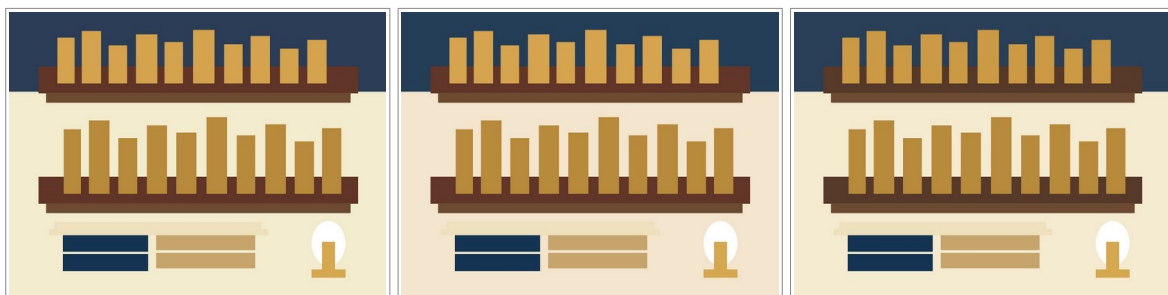
MongoDB no maneja llaves foraneas como SQL, por eso no se aplica cascade, set null ni restrict automatico desde la base de datos. La integridad se maneja desde el formulario, usando listas desplegables que cargan documentos existentes.

Si se elimina un documento relacionado, el sistema no borra automaticamente los demas documentos. Esta decision mantiene el codigo sencillo y facil de entender.

9. Funcionamiento de los Estilos

El diseno usa colores relacionados con una biblioteca: azul oscuro, blanco, beige, cafe claro y detalles dorados. La barra de navegacion se mantiene igual en todas las paginas.

Las tarjetas se usan para mostrar accesos rapidos. Los formularios estan centrados, las tablas tienen encabezados claros y los botones incluyen iconos para que el usuario identifique cada accion facilmente.



10. Uso del Sistema Paso a Paso

1. Entrar a `http://localhost:3000`.
2. Usar la barra de navegacion para elegir autores, libros, usuarios o prestamos.
3. Entrar a una pagina de registro para llenar el formulario.
4. Presionar Guardar para enviar los datos al servidor.
5. Entrar a la pagina de lista para ver los datos guardados.
6. Usar el campo de busqueda para filtrar registros.
7. Usar el boton de lapiz para editar.
8. Usar el boton de basura para eliminar.
9. Usar el boton de impresora para imprimir una lista.

11. CRUD Paso a Paso

Crear

El usuario llena un formulario y presiona Guardar. El navegador manda los datos al servidor con JavaScript. El servidor recibe los datos en una ruta `app.post` y usa `insertOne` para guardar en MongoDB.

Mostrar

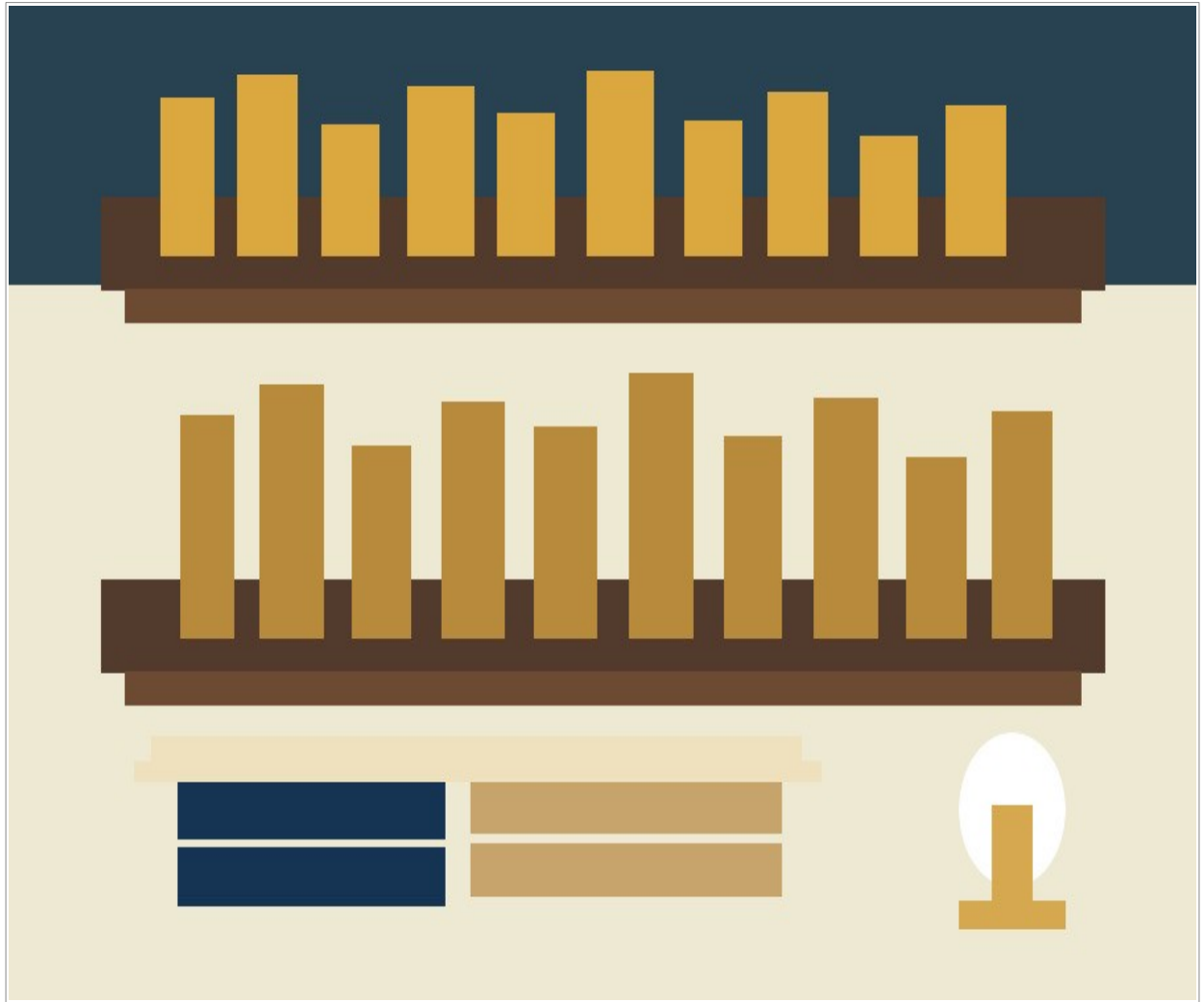
El usuario entra a una lista. La pagina pide datos a una ruta `app.get`. El servidor consulta MongoDB con `find` o `aggregate` y regresa los datos.

Editar y Actualizar

El usuario presiona el boton de lapiz. El sistema abre el formulario con el id del registro, carga la informacion y permite modificarla. Al guardar, el servidor usa `updateOne`.

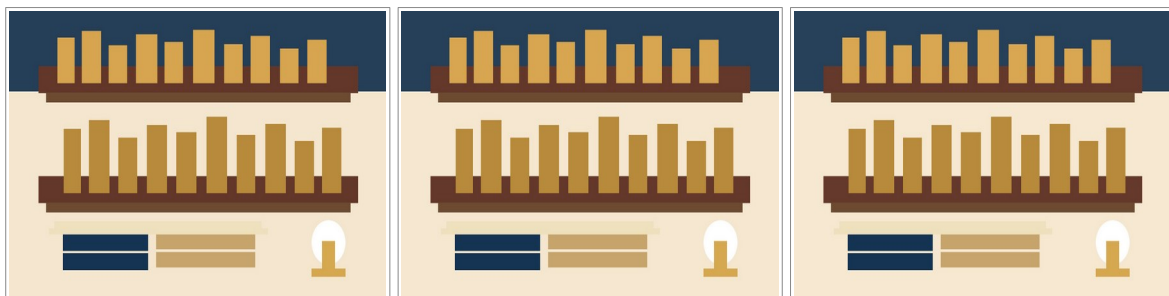
Eliminar

El usuario presiona el boton de basura. `SweetAlert2` pide confirmacion. Si el usuario acepta, el servidor usa `deleteOne` para eliminar el documento.



12. Capturas de Pantalla Simuladas

Las siguientes imagenes se incluyen como capturas simuladas para mostrar el estilo general del proyecto.



13. Validaciones y Manejo de Errores

- Los campos importantes usan required para evitar datos vacios.
- Los correos usan type="email".
- El telefono usa pattern y maxlength para capturar 10 numeros.
- Las fechas usan type="date".
- SweetAlert2 muestra mensajes de exito al guardar, actualizar o eliminar.
- SweetAlert2 pide confirmacion antes de eliminar registros.
- Si ocurre un error de MongoDB, el servidor responde un mensaje y la pagina lo muestra.

14. Lista de Verificacion del Manual

Elemento	Cumple
Incluye portada con nombre del proyecto, alumno, modulo, semestre y fecha.	Si
Incluye indice o estructura del documento.	Si
Describe el proposito del sistema.	Si
Menciona las tecnologias utilizadas: Node y MongoDB.	Si
Explica como ejecutar el software o programa.	Si
Incluye diagrama entidad-relacion adaptado a colecciones NoSQL.	Si
Indica configuracion necesaria de base de datos, servidor y archivos.	Si
Explica el funcionamiento de los estilos.	Si
Describe el uso del sistema y CRUD paso a paso.	Si
Incluye capturas de pantalla claras o simuladas.	Si

Explica la base de datos y sus colecciones.	Si
Describe relaciones NoSQL con \$lookup y aggregate.	Si
Explica restricciones de integridad segun el proyecto.	Si
Menciona validaciones y manejo de errores.	Si
Redaccion clara y sin errores ortograficos importantes.	Si
Documento ordenado y facil de entender.	Si